

# The Ultimate SQL Cheat Sheet

Self-Sufficient Reference Guide for Database Architects & Data Engineers

## PART 1: DDL (Data Definition Language)

*Architectural commands used to structure the database, create tables, and define indexing.*

| Command                   | Full Executable Syntax                                                                         | Detailed Engineering Description                                                                                                                         |
|---------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CREATE DATABASE</b>    | <pre>CREATE DATABASE<br/>enterprise_db;</pre>                                                  | Allocates physical disk space and initiates a new database environment.                                                                                  |
| <b>CREATE TABLE</b>       | <pre>CREATE TABLE users (id<br/>INTEGER PRIMARY KEY, email<br/>TEXT NOT NULL, age INT);</pre>  | Initializes a new table. Explicitly defines column names, strict data types, and primary key constraints to guarantee row uniqueness.                    |
| <b>CTAS (Clone Table)</b> | <pre>CREATE TABLE backup_users<br/>AS SELECT id, email FROM<br/>users WHERE age &gt; 18;</pre> | "Create Table As Select". Instantly generates a brand-new table and populates it with the results of a SELECT query. Excellent for backups.              |
| <b>ALTER TABLE (ADD)</b>  | <pre>ALTER TABLE users ADD<br/>COLUMN phone_number TEXT<br/>DEFAULT 'Unknown';</pre>           | Modifies an existing architecture without dropping data. Appends a new column and automatically fills historical rows with the DEFAULT value.            |
| <b>ALTER TABLE (DROP)</b> | <pre>ALTER TABLE users DROP<br/>COLUMN phone_number;</pre>                                     | Permanently deletes a column and its data. <i>(Note: Support is limited in SQLite, requires table rebuilds, but works flawlessly in Postgres/MySQL).</i> |
| <b>RENAME TABLE</b>       | <pre>ALTER TABLE users RENAME<br/>TO legacy_users;</pre>                                       | Safely changes the table's reference pointer name without touching the underlying hard-drive data.                                                       |

|                       |                                                                                          |                                                                                                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DROP TABLE</b>     | <code>DROP TABLE IF EXISTS legacy_users;</code>                                          | <b>Dangerous.</b> Instantly obliterates the table, its schema, and all data. <code>IF EXISTS</code> prevents the script from crashing if the table is already gone.    |
| <b>TRUNCATE TABLE</b> | <code>TRUNCATE TABLE system_logs;</code>                                                 | Instantly deletes all rows but keeps the empty table structure. Operates via disk unlinking, making it exponentially faster than <code>DELETE FROM</code> .            |
| <b>CREATE INDEX</b>   | <code>CREATE INDEX idx_user_age ON users(age);</code>                                    | Instructs the engine to build a B-Tree structure for the <code>age</code> column. Drastically reduces <code>SELECT</code> time complexity from $O(N)$ to $O(\log N)$ . |
| <b>UNIQUE INDEX</b>   | <code>CREATE UNIQUE INDEX idx_email ON users(email);</code>                              | Builds a B-Tree for speed, while simultaneously forcing the DB to reject any <code>INSERT</code> that contains a duplicate email.                                      |
| <b>DROP INDEX</b>     | <code>DROP INDEX idx_user_age;</code>                                                    | Destroys the B-Tree index. Used when a table is experiencing slow <code>INSERT</code> / <code>UPDATE</code> speeds due to index bloat.                                 |
| <b>CREATE VIEW</b>    | <code>CREATE VIEW adult_users AS SELECT email, age FROM users WHERE age &gt;= 18;</code> | Saves a complex query as a "Virtual Table". It stores the <i>query</i> , not the data. You can now simply run <code>SELECT * FROM adult_users;</code>                  |

## PART 2: DML & DQL (Data Manipulation & Query Language)

*Commands used to insert, mutate, filter, group, and retrieve data payloads.*

| Command | Full Executable Syntax | Detailed Engineering Description |
|---------|------------------------|----------------------------------|
|---------|------------------------|----------------------------------|

|                          |                                                                                                           |                                                                                                                                                                  |
|--------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>INSERT INTO</b>       | <pre>INSERT INTO users (name, age, role) VALUES ('Alice', 25, 'Admin');</pre>                             | Writes a new leaf node to the table. Omitting the <code>id</code> column allows <code>AUTOINCREMENT</code> to automatically assign the next primary key.         |
| <b>INSERT BATCH</b>      | <pre>INSERT INTO users (name) VALUES ('Bob'), ('Charlie'), ('Diana');</pre>                               | Injects multiple rows in a single disk transaction, reducing network latency significantly compared to running multiple individual INSERTs.                      |
| <b>UPSERT (Conflict)</b> | <pre>INSERT INTO users (id, name) VALUES (1, 'Alice') ON CONFLICT(id) DO UPDATE SET name = 'Alice';</pre> | The "Upsert" (Update/Insert). If ID 1 doesn't exist, it inserts Alice. If ID 1 already exists, it updates the row instead of crashing the pipeline.              |
| <b>UPDATE</b>            | <pre>UPDATE users SET role = 'Manager', age = 26 WHERE id = 1;</pre>                                      | Mutates existing data. <b>Always</b> include a <code>WHERE</code> clause; executing <code>UPDATE</code> without one will permanently overwrite the entire table. |
| <b>DELETE</b>            | <pre>DELETE FROM users WHERE last_login &lt; '2020-01-01';</pre>                                          | Deletes rows conditionally. Slower than <code>TRUNCATE</code> because it deletes row-by-row and writes every deletion to the transaction log.                    |
| <b>SELECT</b>            | <pre>SELECT name, age FROM users WHERE role = 'Admin';</pre>                                              | The foundational read query. Avoid <code>SELECT *</code> in production to reduce network I/O and RAM exhaustion.                                                 |
| <b>GROUP BY</b>          | <pre>SELECT role, COUNT(id) FROM users GROUP BY role;</pre>                                               | Compresses data by category. The engine groups identical roles into buckets, then runs the <code>COUNT</code> math inside each isolated bucket.                  |
| <b>HAVING</b>            | <pre>SELECT role, COUNT(id) FROM users GROUP BY role HAVING COUNT(id) &gt; 10;</pre>                      | The filter for aggregated buckets. <code>WHERE</code> filters raw data before grouping; <code>HAVING</code> filters the math buckets after grouping.             |

|                           |                                                                                                                                                                          |                                                                                                                                                                                 |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ORDER BY</b>           | <code>SELECT name, age FROM users<br/>ORDER BY age DESC, name ASC;</code>                                                                                                | Triggers a heap-sort (or index walk). Sorts primarily by Age (High to Low), and breaks ties alphabetically by Name.                                                             |
| <b>LIMIT &amp; OFFSET</b> | <code>SELECT name FROM users ORDER<br/>BY id ASC LIMIT 10 OFFSET 50;</code>                                                                                              | Core mechanic for UI Pagination (e.g., "Page 6"). Skips the first 50 rows, then extracts the exact next 10 rows.                                                                |
| <b>INNER JOIN</b>         | <code>SELECT u.name, p.price FROM<br/>users u INNER JOIN purchases p<br/>ON u.id = p.user_id;</code>                                                                     | Intersection. Scans both tables and only returns a row if the <code>user_id</code> perfectly matches on both sides. Users with no purchases are dropped.                        |
| <b>LEFT JOIN</b>          | <code>SELECT u.name, p.price FROM<br/>users u LEFT JOIN purchases p<br/>ON u.id = p.user_id;</code>                                                                      | Base Set Preservation. Guarantees every user from the left table ( <code>u</code> ) is printed. If a user lacks a purchase, <code>p.price</code> returns as <code>NULL</code> . |
| <b>FULL OUTER JOIN</b>    | <code>SELECT u.name, p.price FROM<br/>users u FULL OUTER JOIN<br/>purchases p ON u.id =<br/>p.user_id;</code>                                                            | The Mathematical Union. Merges everything. If a user bought nothing, price is <code>NULL</code> . If an orphaned purchase has no user, name is <code>NULL</code> .              |
| <b>UNION ALL</b>          | <code>SELECT email FROM customers<br/>UNION ALL SELECT email FROM<br/>employees;</code>                                                                                  | Vertically stacks two separate queries into one single column output. <code>UNION ALL</code> keeps duplicate emails; standard <code>UNION</code> drops duplicates.              |
| <b>CTE (WITH)</b>         | <code>WITH rich_users AS (SELECT id<br/>FROM users WHERE balance &gt; 1M)<br/>SELECT p.item FROM purchases p<br/>INNER JOIN rich_users r ON<br/>p.user_id = r.id;</code> | Common Table Expression. Creates a clean, temporary variable query ( <code>rich_users</code> ) that can be executed and joined immediately in the main query.                   |



# PART 3: DCL & TCL (Data Control & Transaction Control)

Security protocols and ACID-compliant hardware transaction states.

| Command   | Full Executable Syntax                                 | Detailed Engineering Description                                                                                                                |
|-----------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| GRANT     | GRANT SELECT, INSERT ON users TO 'api_backend_role';   | Gives a specific user, app, or role the exact read/write permissions they need, blocking them from DELETE or DROP commands.                     |
| REVOKE    | REVOKE INSERT ON users FROM 'api_backend_role';        | Strips privileges, immediately downgrading the targeted role to read-only access for that specific table.                                       |
| BEGIN     | BEGIN TRANSACTION;                                     | Suspends automatic saving. Opens a temporary memory sandbox. No data written here is visible to other users until finalized.                    |
| COMMIT    | COMMIT;                                                | Triggers an OS-level file sync. Flushes the temporary transaction sandbox to the permanent SSD. The changes are now locked in.                  |
| ROLLBACK  | ROLLBACK;                                              | The Emergency Abort. If an error occurs halfway through a script, this deletes the temporary sandbox, leaving the database perfectly untouched. |
| SAVEPOINT | SAVEPOINT step1; DELETE FROM users; ROLLBACK TO step1; | Creates checkpoints within a massive transaction. Allows you to undo a specific mistake without having to rollback the entire transaction.      |



# PART 4: SQL Functions Library

C-level computational functions executed directly adjacent to the hard drive.



## A. Aggregate Functions (Vertical Dataset Computations)

| Function     | Full Executable Syntax                                                                                  | Detailed Engineering Description                                                                                                                     |
|--------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| COUNT        | <pre>SELECT COUNT(id),<br/>COUNT(email) FROM users;</pre>                                               | <p>COUNT(id) counts all rows.</p> <p>COUNT(email) counts all rows <i>except</i> those where email is NULL.</p>                                       |
| SUM & AVG    | <pre>SELECT SUM(price),<br/>AVG(price) FROM purchases;</pre>                                            | Computes Total and Mathematical Mean. Drastically reduces network payload sizes compared to Python looping.                                          |
| MIN & MAX    | <pre>SELECT MIN(age),<br/>MAX(created_at) FROM users;</pre>                                             | Extracts boundary limits. Operates in $O(1)$ time if targeting an indexed column. MAX on dates finds the newest entry.                               |
| GROUP_CONCAT | <pre>SELECT department,<br/>GROUP_CONCAT(name, ', ' )<br/>FROM employees GROUP BY<br/>department;</pre> | <p>(Called STRING_AGG in Postgres).</p> <p>Compresses a vertical column of names into a single horizontal comma-separated string per department.</p> |



## B. Window Functions (Advanced Data Science)

| Function        | Full Executable Syntax                                                                           | Detailed Engineering Description                                                                                                                                   |
|-----------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OVER(PARTITION) | <pre>SELECT name, price,<br/>AVG(price) OVER(PARTITION<br/>BY category) FROM<br/>products;</pre> | Performs an aggregation (Average) grouped by category, <i>without</i> collapsing the rows. Each row displays the product's price alongside its category's average. |
| ROW_NUMBER()    | <pre>SELECT name, ROW_NUMBER()<br/>OVER(ORDER BY score DESC)<br/>AS rank FROM users;</pre>       | Dynamically assigns a sequential integer (1, 2, 3...) to every row based on the sorting condition.                                                                 |

|               |                                                                                                              |                                                                                                                                                |
|---------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RANK()</b> | <pre>SELECT name, RANK()<br/>OVER(ORDER BY score DESC)<br/>FROM users;</pre>                                 | Like Row Number, but respects ties. If User A and B tie for 1st place, they both get 1, and User C drops down to rank 3.                       |
| <b>LAG()</b>  | <pre>SELECT date, revenue,<br/>LAG(revenue, 1) OVER(ORDER<br/>BY date) AS prev_day_rev<br/>FROM sales;</pre> | Cross-row reference. Grabs the data from the <i>previous</i> row and places it in the current row. Essential for Day-over-Day growth tracking. |
| <b>LEAD()</b> | <pre>SELECT name, LEAD(name,<br/>1) OVER(ORDER BY<br/>turn_order) AS next_player<br/>FROM game;</pre>        | Cross-row reference. Grabs the data from the <i>next</i> row down the table.                                                                   |

## C. Scalar / String Functions (Row-Level Manipulations)

| Function           | Full Executable Syntax                                                                | Detailed Engineering Description                                                                               |
|--------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>CONCAT()</b>    | <pre>SELECT CONCAT(first_name, ' ',<br/>last_name) AS full_name FROM<br/>users;</pre> | Safely stitches strings together. Avoids the + or `                                                            |
| <b>SUBSTRING()</b> | <pre>SELECT SUBSTRING(phone_number,<br/>1, 3) AS area_code FROM users;</pre>          | Extracts a slice of a string. Extracts 3 characters, starting exactly from character position 1.               |
| <b>REPLACE()</b>   | <pre>SELECT REPLACE(phone_number, '-<br/>, ''') FROM users;</pre>                     | Data scrubbing. Scans the string, finds every dash -, and replaces it with nothing ''.                         |
| <b>UPPER/LOWER</b> | <pre>SELECT * FROM users WHERE<br/>LOWER(email) = 'ada@test.com';</pre>               | Forces string normalization. Prevents query failure if the user signed up using <a href="#">ADA@test.com</a> . |

|        |                                          |                                                                                                                   |
|--------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| TRIM() | <pre>SELECT TRIM(name) FROM users;</pre> | Eradicates invisible whitespace (e.g., spaces, tabs) accidentally saved at the very beginning or end of a string. |
|--------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|

## D. Date, Time, & Formatting Functions

| Function        | Full Executable Syntax                                                                              | Detailed Engineering Description                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| NOW() / CURRENT | <pre>UPDATE users SET last_login = CURRENT_TIMESTAMP WHERE id = 1;</pre>                            | Automatically grabs the server's exact UTC hardware clock time and stamps it to the database.                        |
| EXTRACT()       | <pre>SELECT EXTRACT(MONTH FROM purchase_date) AS month, SUM(price) FROM sales GROUP BY month;</pre> | Dissects a timestamp ( 2024-05-12 ), isolates the specific component (Month 05 ), and groups revenue by it.          |
| DATEADD()       | <pre>SELECT DATEADD(day, 30, trial_start_date) AS billing_date FROM accounts;</pre>                 | Date math. Automatically calculates exactly 30 days into the future, correctly respecting leap years and month ends. |
| DATEDIFF()      | <pre>SELECT DATEDIFF(day, sign_up_date, CURRENT_TIMESTAMP) AS account_age_days FROM users;</pre>    | Returns the absolute integer difference (in days, hours, or months) between two timestamps.                          |

## E. Advanced Flow Control, Casts, & JSON

| Function  | Full Executable Syntax                                                                                   | Detailed Engineering Description                                                                                           |
|-----------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| CASE WHEN | <pre>SELECT name, CASE WHEN age &gt;= 18 THEN 'Adult' ELSE 'Minor' END AS legal_status FROM users;</pre> | The SQL standard If/Elif/Else statement. Evaluates row data on the fly and generates a dynamically computed output string. |



|                     |                                                                                                     |                                                                                                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>COALESCE()</b>   | <pre>SELECT name, COALESCE(phone, email, 'No Contact Provided') AS contact_method FROM users;</pre> | <p>Evaluates parameters left-to-right. It stops and returns the <i>very first</i> value that is NOT <code>NULL</code> . Perfect for UI fallback logic.</p>            |
| <b>NULLIF()</b>     | <pre>SELECT revenue / NULLIF(total_clicks, 0) AS revenue_per_click FROM ad_data;</pre>              | <p>Returns <code>NULL</code> if total clicks is 0. This purposefully prevents the query from suffering a catastrophic "Divide by Zero" crash.</p>                     |
| <b>CAST()</b>       | <pre>SELECT CAST(price_string AS REAL) FROM raw_import;</pre>                                       | <p>Forcibly converts a data type. Casts <code>'99.99'</code> (Text) into <code>99.99</code> (Floating point integer) so mathematics can be performed.</p>             |
| <b>JSON_EXTRACT</b> | <pre>SELECT JSON_EXTRACT(server_logs, '\$.browser_type') FROM logs;</pre>                           | <p>Reaches into a raw JSON text dump ( <code>{"browser_type": "Chrome", "IP": "..."} </code> ), extracts specific dictionary keys, and acts like normal SQL data.</p> |